



Design and Verification of UART (Universal Asynchronous Receiver Transmitter)

SUBMITTED BY:

Name: Koneti Ajay

Batch: Hybrid

Email: ajjikoneti@gmail.com

INDEX

1. Introduction
2. UART Architecture
3. RTL Design
4. Testbench Environment
5. Simulation Results
6. Conclusion

INTRODUCTION:

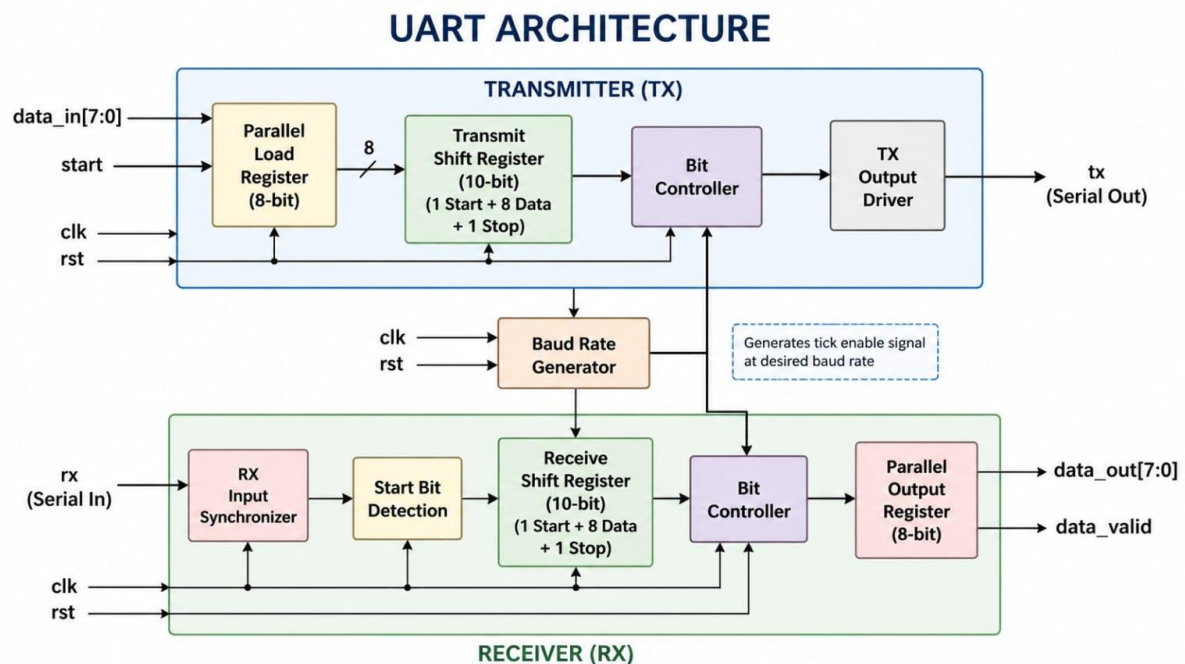
UART (Universal Asynchronous Receiver Transmitter) is a communication protocol used for serial data transmission. It converts parallel data into serial form for transmission and reconstructs it back at the receiver.

- It is widely used in embedded systems, microcontrollers, and communication interfaces.

UART ARCHITECTURE:

The UART consists of:

- Transmitter (TX)
- Receiver (RX)
- Baud Rate Generator
- Shift Registers



RTL DESIGN:

The RTL Design consists of 4 codes along with testbench code.

The codes are:

- Transmitter Code
- Receiver Code
- Top Module
- Test Bench Code

Verilog Code for Transmission:

```

module uart_tx (
    input clk, rst, start,
    input [7:0] data_in,
    output reg tx,
    output reg busy
);

parameter BAUD_COUNT = 5208;

reg [12:0] baud_cnt;
reg baud_tick;
reg [3:0] bit_cnt;
reg [9:0] shift_reg;

// ----- BAUD GENERATOR -----
always @(posedge clk or posedge rst) begin
    if (rst) begin
        baud_cnt <= 0;
        baud_tick <= 0;
    end
    else if (busy) begin
        if (baud_cnt == BAUD_COUNT-1) begin
            baud_cnt <= 0;
            baud_tick <= 1;
        end else begin
            baud_cnt <= baud_cnt + 1;
            baud_tick <= 0;
        end
    end
    else begin
        baud_cnt <= 0;
        baud_tick <= 0;
    end
end

// ----- TRANSMITTER -----

always @(posedge clk or posedge rst) begin
    if (rst) begin
        tx <= 1;
        busy <= 0;
        bit_cnt <= 0;
        shift_reg <= 0;
    end
    else begin
        // Start transmission
        if (start && !busy) begin
            shift_reg <= {1'b1, data_in, 1'b0}; // stop + data + start
            busy <= 1;
            bit_cnt <= 0;
        end
        // Transmitting bits
    end
end

```

```

        else if (busy && baud_tick) begin
            tx <= shift_reg[0];
            shift_reg <= shift_reg >> 1;
            bit_cnt <= bit_cnt + 1;

            if (bit_cnt == 9) begin // 10 bits total
                busy <= 0;
                tx <= 1; // idle state
            end
        end
    end
end
endmodule

```

Verilog Code for Receiver:

```

module uart_rx (
    input clk,
    input rst,
    input rx,
    output reg [7:0] data_out,
    output reg valid
);
parameter BAUD_COUNT = 5208;
reg [12:0] baud_cnt;
reg baud_tick;
reg [3:0] bit_cnt;
reg [1:0] state;
reg [7:0] data_reg;
parameter IDLE=0, START=1, DATA=2, STOP=3;
// ----- BAUD -----
always @(posedge clk or posedge rst) begin
    if (rst) begin
        baud_cnt <= 0;
        baud_tick <= 0;
    end
    else if (state != IDLE) begin

```

```

    if (baud_cnt == BAUD_COUNT-1) begin
        baud_cnt <= 0;
        baud_tick <= 1;
    end else begin
        baud_cnt <= baud_cnt + 1;
        baud_tick <= 0;
    end
end
else begin
    baud_cnt <= 0;
    baud_tick <= 0;
end
end

// ----- FSM -----
always @(posedge clk or posedge rst) begin
    if (rst) begin
        state <= IDLE;
        valid <= 0;
        bit_cnt <= 0;
    end
    else begin
        case(state)
        IDLE: begin
            valid <= 0;
            if (rx == 0) begin
                state <= START;
                baud_cnt <= BAUD_COUNT/2; // mid sampling
            end
        end
    end

    START: begin

```

```

    if (baud_tick) begin
        if (rx == 0) begin
            state <= DATA;
            bit_cnt <= 0;
        end else
            state <= IDLE;
        end
    end
end

DATA: begin
    if (baud_tick) begin
        data_reg[bit_cnt] <= rx;
        bit_cnt <= bit_cnt + 1;
        if (bit_cnt == 7)
            state <= STOP;
        end
    end
end

STOP: begin
    if (baud_tick) begin
        if (rx == 1) begin
            data_out <= data_reg;
            valid <= 1;
        end
        state <= IDLE;
    end
end
endcase
end
end
endmodule

```

Verilog Code for Top Module:

```

module uart_top (
    input clk,
    input rst,
    input start,
    input [7:0] data_in,
    output [7:0] data_out,
    output valid,
    output tx_busy
);
    // Internal wire for loopback connection
    wire tx_line;
    // ----- TX -----
    uart_tx tx_inst (
        .clk(clk),
        .rst(rst),
        .start(start),
        .data_in(data_in),
        .tx(tx_line),
        .busy(tx_busy)
    );
    // ----- RX -----
    uart_rx rx_inst (
        .clk(clk),
        .rst(rst),
        .rx(tx_line),    // loopback connection
        .data_out(data_out),
        .valid(valid)
    );

endmodule

```

TESTBENCH FOR UART:

A Verilog testbench is used to apply test inputs and verify UART behavior under different scenarios.

Testbench Code using System Verilog:

```
`timescale 1ns/1ps

module uart_tb;

logic clk = 0, rst, start;
logic [7:0] data_in, data_out;
logic valid, tx_busy;

// DUT
uart_top dut (.*);

// Clock
always #10 clk = ~clk;

// Send task
task send(input [7:0] d);
begin
    @(posedge clk);
    while (tx_busy) @(posedge clk);
    data_in = d;
    start = 1;
    @(posedge clk);
    start = 0;
end
endtask

// Test
initial begin
    rst = 1; start = 0; data_in = 0;

    repeat(5) @(posedge clk); // hold reset for 5 clock cycles
    rst = 0;
    send(8'hA5); wait(valid);
    send(8'h3C); wait(valid);
    send(8'hF0); wait(valid);

    repeat(3) begin
        send($urandom_range(0,255));
        wait(valid);
    end

    #200 $finish;
end

// Display result
```

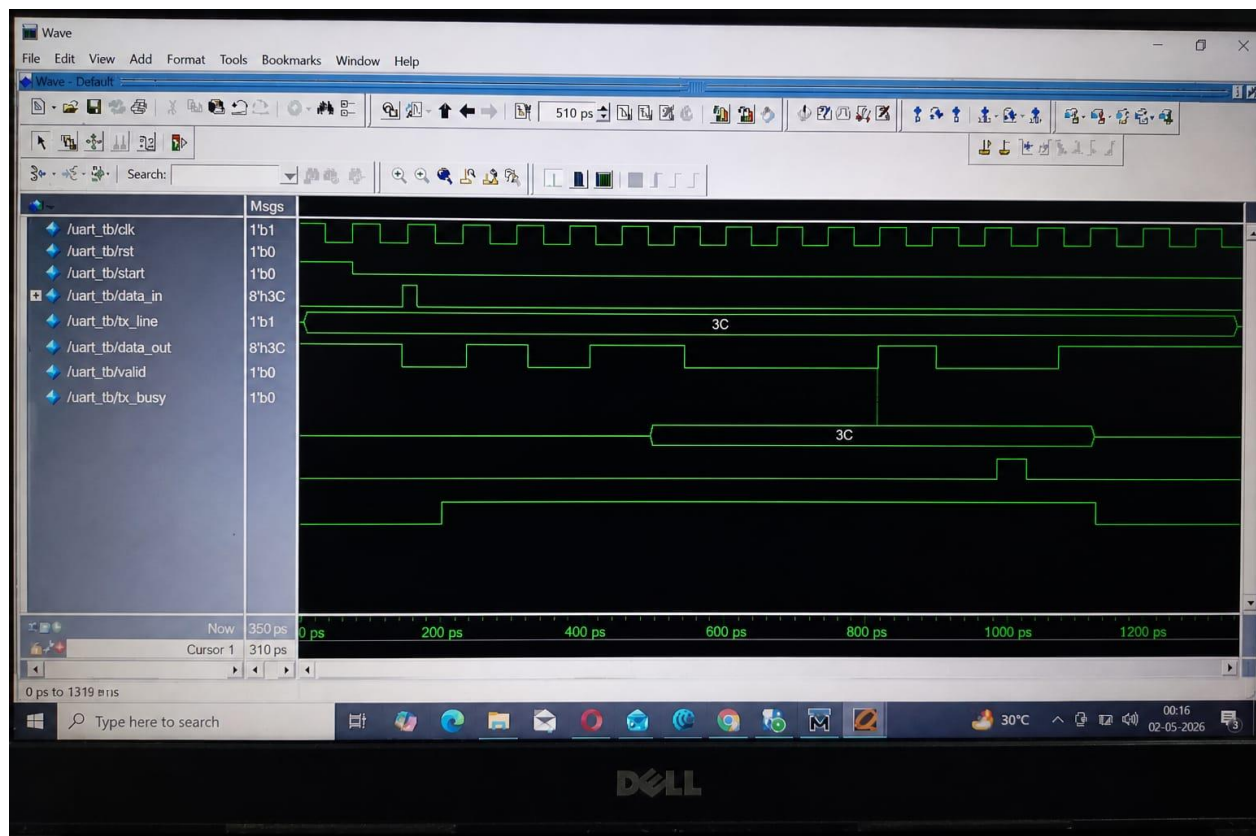
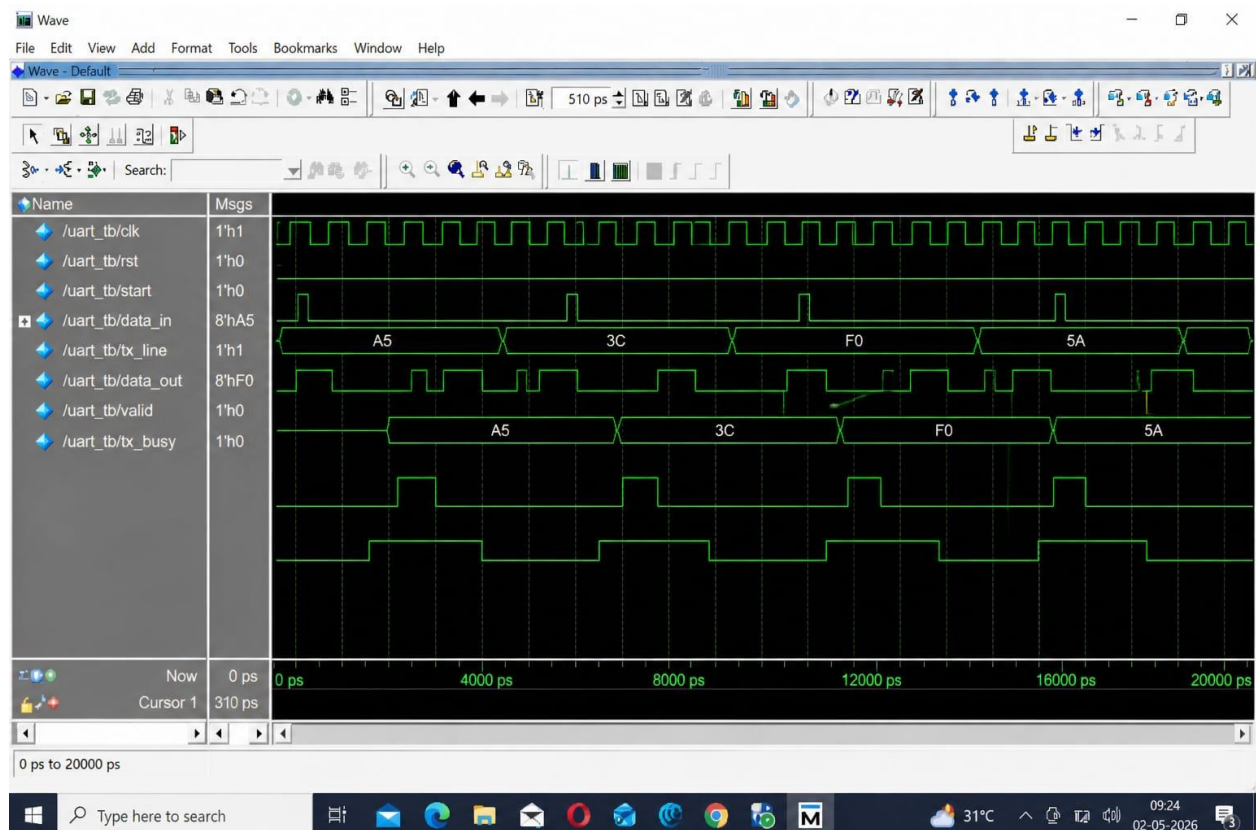
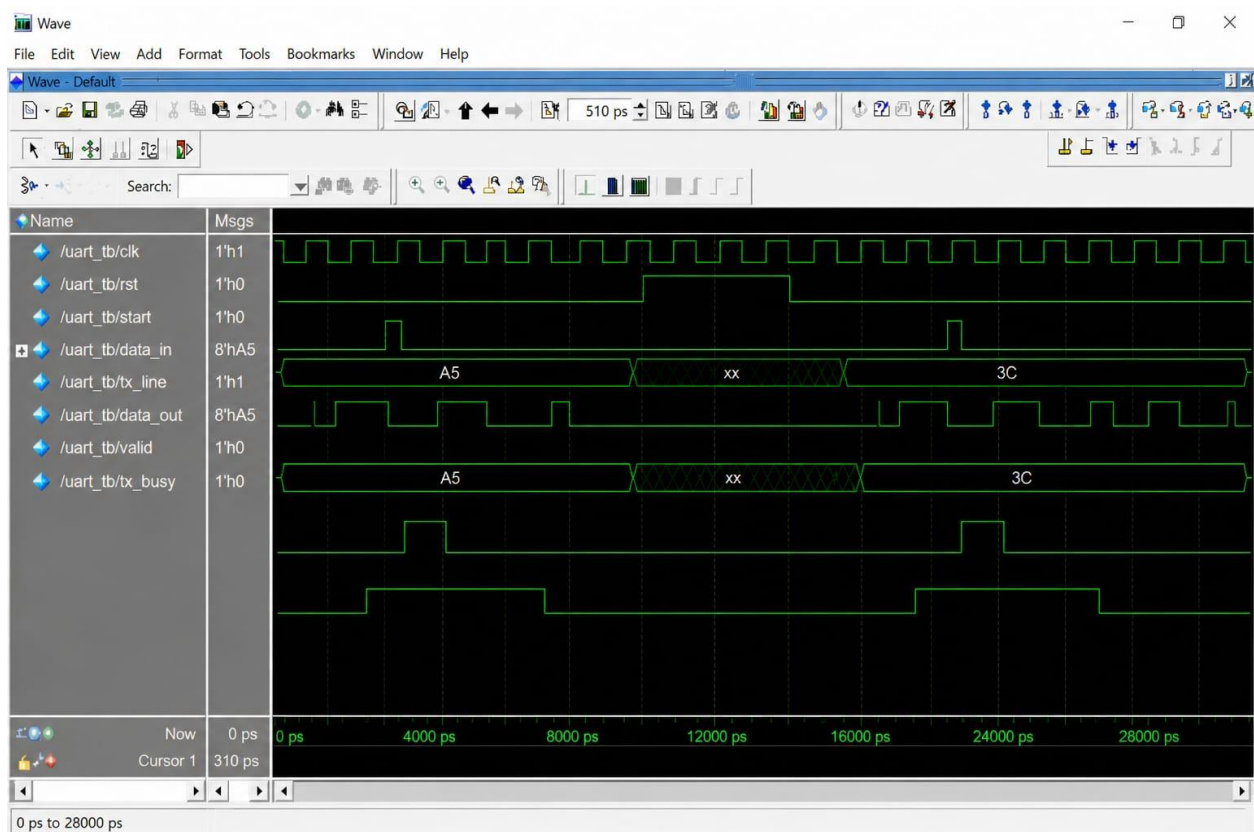

FIGURE 2: Back-To-Back Transmission**FIGURE 3: Random Data Transmission**

FIGURE 4: Reset During Transmission

CONCLUSION:

The UART design was successfully implemented and verified. All test cases showed correct data transmission and reception. The project demonstrates effective understanding of serial communication and Verilog-based design.